

CineCart: Polymorphism & Inheritance Extension

Object-Oriented Programming — Lab 4 Extension II

Marks: 10 **Language:** Java (JDK 17) **Approach:** Test-Driven

This extension follows the Lab 4 core and the *Self-Defending Classes* addendum. Submit your .java source files only.

The restriction we made in Lab 4 is now lifted

At the end of Lab 4 a notice told you: “Resist the urge to introduce inheritance, interfaces, or design patterns; you have not been taught those yet, and this lab does not need them.” You have now been taught the first two pillars they referred to. This extension removes that restriction and, in fact, **requires** you to use what you have learned. The booth’s price list has grown, and the flat, copy-the-arguments design of Lab 4 has started to creak. Your job is to let the right objects carry their own behaviour, so that the till can treat very different things *uniformly*.

What to Submit

You will continue working in the same CineCart/ Gradle project. Submit your .java source files only. The grader will run `./gradlew test` against a new test file:

- `src/test/java/CineCartPolymorphismTest.java` — shipped with this extension. **Every test must pass.**

► Notice

The Lab 4 constraints still hold: do *not* modify `CsvLoader.java`, the CSV files, or `build.gradle`. Do not import `java.util.ArrayList`, `List`, `HashMap`, `Arrays`, or any other collection or helper class. **Fixed-size arrays only**, each tracked by its own `int` count — exactly as before.

This extension *redesigns* part of the model, so some Lab 4 internals will change shape. The new suite re-checks every pricing rule, so the numbers your booth produces must not drift; what changes is *where* those numbers are computed and *which object* is responsible for each one.

Where Lab 4 Now Strains

Look at your Lab 4 solution before you start. After you read the new business rules 3 things should feel out of place:

- The Cart keeps *two* arrays — `Ticket[]` and `ConcessionItem[]` — and checkout sums each with its *own* loop. The accountant below wants *one* walk over *one* collection.
- Every ticket is priced by the *same* formula, baked into `CheckoutEngine.bookTicket`. The price list below has *four* different tickets, each with its own rule, and a snack counter that now prices things three different ways.
- `CheckoutEngine` decides everything; the model objects decide nothing. Below, the *things* must start deciding their own charge.

The Scenario: CineCart Grows Up

Cineplex Bashundhara has done well, and the price list has grown in three directions at once. Read these business rules slowly. Each one is a fact about the world; together they decide how your classes must relate. Nothing here tells you *which language feature* to reach for — that is the part you are meant to work out.

The tickets are no longer all alike

- Tickets now come in several **fare classes**: a *standard* seat, a *premium* recliner, a *student* concession fare, and a *VIP* recliner-with-lounge. Every one of them is, unmistakably, *a kind of* ticket — each occupies a seat at a showtime and is sold at the same booth — yet each works out its charge by its *own* rule.
- There is no neutral, rule-less ticket that anyone could ever sell. *A bare ticket with no fare rule of its own should never come into existence.* If a colleague tries to bring one into being, the code should not even let them.
- The **VIP** fare is not a brand-new invention. It *is* the premium recliner fare *plus* a flat lounge charge. Whatever the premium fare works out to, the VIP fare *begins from that figure* and adds to it. If the premium recliner rule is ever re-priced, the VIP fare must move with it *for free*, without anyone editing the VIP code.

The snack counter prices things three ways

- It still sells a single menu item by *quantity* (three popcorns).
- It now also sells a named **bundle** — a popcorn-and-soda meal — at a single set-aside price. A bundle is built *out of* the menu items it contains; it is plainly *not* itself a popcorn, and it would be absurd for the till to treat it as one.

What the accountant demands of the till

- **One walk, one total.** The cart must hold *every* chargeable thing — tickets and snacks alike — and the checkout must total them in a *single pass*: “for each chargeable thing in the cart, add what it costs.” Crucially, that loop must contain *no branch that asks what kind of thing each line is.* From the till’s point of view all of these things must look the *same*: one variable must be able to *stand in for any* chargeable line, and one call name must mean “tell me your charge,” whoever is answering.
- **The base figure stays in the family.** Each ticket starts its sum from the showtime’s published base price. That figure must be *directly reachable by the fare-class code that needs it*, yet *must stay hidden from any code outside the ticket family* — an outsider must never be able to reach in and reset it to a nonsense value. It is neither fully open nor fully sealed; it is shared with descendants and with no one else.
- **Receipts grow, they do not get rewritten.** Each chargeable line can say what it is on the receipt. Some lines — the VIP fare, the student fare — describe themselves by *taking the ordinary description and adding to it*, not by writing a fresh description from scratch.
- **One name, several ways to ask.** The till should let the counter add a snack in more than one *legitimate way*: a full form that states a quantity, and a short form for the common single-item case. Same name, same idea, different inputs — the reader should not have to memorise a family of near-identical method names.

A note on reading these rules. Every phrase in *italics* above is load-bearing. “A kind of,” “should never come into existence,” “begins from that figure and adds to it,” “stand in for any,” “reachable by ... hidden from outside,” “adding to it, not rewriting”—each describes a relationship between classes, and each relationship has exactly one natural way to express it in Java. Decide the relationships *first*, on paper, before you write a single class. If you start by typing, you will guess wrong.

Part H — Many Forms of a Cart

[10 marks]

The tasks below describe *what must be true*. They do not tell you how to arrange your classes; that is the assessed part. The contract that the test file depends on is pinned in “The New Test File” section — read it together with these tasks, because the names and signatures the tests call are fixed even though the relationships between your classes are yours to design.

H.1 A single kind of “chargeable line,” and a one-pass total

Today the cart distinguishes a Ticket from a ConcessionItem at every turn. Replace that split at the level of the cart’s storage: the cart must hold a *single* collection (one fixed-size array, one count) of chargeable things, into which an ticket and a snack are equally welcome.

For the checkout to total that collection in one pass with no kind-test inside the loop, every member of the collection must answer the *same* question — “what do you cost?” — under the *same* name. Provide:

- a Cart that stores all chargeable lines in one array and exposes `LineItem[] getLines()` returning a *defensive snapshot* (length equal to the live count, writing into it must not corrupt the cart — the same discipline you learned in the Self-Defending addendum);
- `double grandSubtotal()` on Cart, computed by a *single* loop over that one array, with no test of any line’s specific type inside the loop;
- the means for `CheckoutEngine.checkout` to obtain the pre-discount subtotal from that one method.

Question Your Viva Will Ask (H.1)

Your `grandSubtotal()` loop calls one method name on each element, yet a standard fare, a VIP fare, and a snack bundle each run *different* arithmetic. The loop was compiled before those classes were even finalised in your mind. *Who* chooses which arithmetic runs for a given element, and at *what moment* — while you compile, or while the program runs? Name the moment precisely.

H.2 The ticket family: one idea, four rules

Model the four fare classes so that all of them are interchangeable wherever a chargeable line is expected, and each computes its own charge. With base the showtime’s published price and a *peak* surcharge of $\times 1.20$ applying when `showtime.isPeak()` (and $\times 1.00$ otherwise):

Fare class	Charge
StandardTicket	$\text{base} \times \text{peak}$
PremiumTicket	$\text{base} \times 1.30 \times \text{peak}$
StudentTicket	$\text{base} \times 0.50 \times \text{peak}$
VIPTicket	(the premium charge) + 200.00 flat lounge

The shared state every fare needs — the showtime, the seat’s row and column, and the base figure — belongs in *one* place, written once, not copied into four classes. That shared figure must obey the visibility rule from the scenario: reachable by the fare code, sealed against the outside.

It must be *impossible* to construct a fare-less, rule-less ticket. Whatever common thing your four fares share, no one should be able to instantiate it on its own.

Question Your Viva Will Ask (H.2)

You wrote the showtime, row, column, and base figure exactly once, in the one ticket type all four fares share, yet a freshly built `StudentTicket` clearly has all four. Which constructor set those fields, and what is the *first* statement that every fare-class constructor must execute before it does anything of its own? What happens at compile time if you forget it?

H.3 VIP is a special premium (and reuses it)

The VIP fare must be modelled as a *specialisation of the premium fare*, not as an independent class that happens to know the premium formula. Concretely: `VIPTicket`’s charge must be obtained by *asking the premium fare for its charge and adding the lounge surcharge to the result* — so that if you later change the premium multiplier from 1.30 to anything else, `VIPTicket` changes automatically, with no edit to `VIPTicket`’s own arithmetic.

Likewise, `VIPTicket`’s receipt description must be the premium/standard description *with the lounge noted on top of it*, produced by reusing the inherited description rather than re-deriving it.

Question Your Viva Will Ask (H.3)

`VIPTicket` is a kind of `PremiumTicket`, which is a kind of ticket. When you call the charge on a `VIPTicket`, Java has to find *which* class’s version to run. From which class does that search start, and in which direction does it walk the chain — and why would searching from the other end give the wrong answer once VIP has added its surcharge?

H.4 Snacks become lines — and the bundle is not a popcorn

Two kinds of snack line must also be chargeable lines, interchangeable with tickets in the cart’s single collection:

- `ConcessionLine` — a single menu item bought in some quantity; its charge is $\text{unitPrice} \times \text{qty}$.
- `ComboLine` — a bundle built *out of* two menu items, charged at $(\text{unitPriceA} + \text{unitPriceB}) - 50.00$.

Think carefully about how `ComboLine` relates to the items it bundles. A bundle *contains* a popcorn and a soda; it is not *a kind of* popcorn. Say each relationship aloud before you commit to it. One of the relationships in this whole extension is ownership and the rest are kinship — mixing them up is the single most common way this lab goes wrong.

Question Your Viva Will Ask (H.4)

Suppose a careless teammate wrote `ComboLine` extends `ConcessionLine` “to reuse the quantity field.” State the sentence the type system would then believe about every `ComboLine`, and give one concrete call that would compile but make no sense. Then state the relationship that is actually true, in “*has-a*” / “*is-a*” form.

H.5 One name, several inputs

The cart must let a caller add a snack in two ways that share *one* method name:

- the full form that states a menu item and a quantity;
- a short form that takes a menu item alone and means “quantity one.”

Both must end with the same effect: a chargeable snack line in the cart. The decision about which form a given call site uses is made entirely from what the caller wrote, before the program runs.

Question Your Viva Will Ask (H.5)

This “one name, several inputs” is the *same* idea as the two Customer constructors you wrote in Lab 4 — and a *different* idea from the four fares all answering one charge call. Name both ideas. Which one is decided by the compiler from the call site, and which by the running program from the live object? What single question separates them?

H.6 The checkout pipeline, re-expressed over lines

CheckoutEngine.checkout(Cart) must produce exactly the same final number as Lab 4 did, but now driven by the unified collection. Apply, in order:

1. subtotal = cart.grandSubtotal() (the single pass of H.1).
2. Combo deal: if the cart’s snack lines together include *both* a POP and a SODA (in any line, single or bundled), subtract 50.0; else 0.0.
3. preDiscount = subtotal – combo.
4. Group discount: if the cart holds *four or more ticket lines*, subtract $0.10 \times \text{preDiscount}$.
5. Tier discount: subtract `cart.getOwner().getTierDiscount()` $\times$ preDiscount.
6. Tax: add $0.05 \times (\text{afterDiscounts})$.
7. Return the result rounded to two decimals.

Notice that steps 2 and 4 *do* need to know what kind a line is — but they are outside the summation loop of H.1, which must stay kind-free. Deciding *how* the cart answers “how many tickets do you hold?” and “do your snack lines include this code?” without re-introducing a forest of type-tests is part of the design you are being assessed on.

Question Your Viva Will Ask (H.6)

The summation loop in H.1 forbids asking a line “what kind are you?”, yet the group and combo rules clearly need that information. Explain why one of these is the *right* place for runtime polymorphism and the other is a legitimate place to ask about type — and propose a way to let each line answer “am I an ticket?” / “do I carry code X?” that keeps the knowledge *inside* each line rather than in a growing if-ladder in the engine.

The New Test File

Your instructor will drop `CineCartPolymorphismTest.java` into `src/test/java/`. You do not write it or modify it; you make every test pass. The contract below is the *only* part of your design that is fixed — the names and signatures the tests call. **How these types relate to one another is deliberately not specified here; deriving the relationships from the scenario is the assessed work.**

The fixed public surface (what the tests construct and call)

```
// Each fare is built from a showtime, a row, and a column:
new StandardTicket(showtime, row, col)
new PremiumTicket (showtime, row, col)
new StudentTicket (showtime, row, col)
new VIPTicket      (showtime, row, col)

// Each snack line:
new ConcessionLine(item, qty)      // item is a ConcessionItem
new ComboLine(itemA, itemB)        // two ConcessionItems

// Every one of the six types above answers these, under one common type:
double subtotal();                 // its own charge
String describe();                 // its receipt line

// A variable of the common type can hold any of the six:
LineItem line = new VIPTicket(showtime, 2, 4);

// Cart, over the single collection:
cart.add(line);                    // add any chargeable line
cart.add(item, qty);               // overloaded: snack convenience
cart.add(item);                    // overloaded: quantity one
LineItem[] getLines();             // defensive snapshot, length == count
double grandSubtotal();            // single pass
int ticketCount();                 // how many lines are tickets
boolean hasCode(String code);      // any snack line carrying this code
```

What the tests check

- `standardFare_isBaseTimesPeak`, `premiumFare_is130PercentOfBase`, `studentFare_isHalfBase` — each fare's `subtotal()` against its rule, at peak and off-peak showtimes.
- `vipFare_equalsPremiumPlusLounge` — for the *same* showtime and seat, `vip.subtotal() == premium.subtotal() + 200.0`. (A later test re-prices the premium multiplier via a fare built at a different base and confirms VIP still tracks it.)
- `vipDescription_extendsPremiumDescription` — `vip.describe()` *contains* the premium/standard description as a substring, plus a lounge marker.
- `concessionLine_isUnitPriceTimesQty`, `comboLine_isPairMinusFifty`.
- `cart_holdsMixedLines_inOneCollection` — a mix of fares and snack lines added to one cart; `getLines().length` equals the number added.
- `cart_getLines_isDefensiveCopy` — writing null into the returned array leaves the cart intact.
- `cart_grandSubtotal_equalsSumOfEachLinesOwnRule` — the grand subtotal of a mixed cart equals the hand-summed total of each element's `subtotal()`: the proof that the right rule ran for each live object.

- `cart_add_isOverloaded` — `add(item)` and `add(item, 1)` produce equal-charged lines; `add(item, 3)` charges triple.
- `cart_ticketCount_countsOnlyTickets`, `cart_hasCode_findsBundledAndSingleSnacks`.
- `checkout_total_matchesLab4Pipeline` — an end-to-end cart prices identically to the Lab 4 rules.

▷ Notice

Some properties cannot be expressed as a JUnit assertion — you *cannot* write `assertThrows` on `new <common type>()` if your design makes that a compile error, which is exactly what the scenario demands. Those properties (the rule-less ticket being impossible to instantiate; the base figure being sealed against outside code; the bundle modelled as ownership not kinship) are checked by **inspection and viva**, and carry marks in the rubric below. Passing the tests is necessary but not sufficient.

Marking Rubric

Part	Criterion	Marks
H.1	One unified collection; kind-free single-pass <code>grandSubtotal()</code> ; defensive <code>getLines()</code>	3
H.2	Four fares interchangeable as one type, each its own rule; shared state written once and correctly sealed; rule-less ticket cannot be instantiated	2
H.3	VIP modelled as a specialisation of premium; charge <i>and</i> description reuse the premium versions rather than copying them	1.5
H.4	Snack lines interchangeable with tickets; bundle modelled as ownership, not kinship	1.5
H.5	Overloaded snack-add: one name, two input shapes, resolved at the call site	1
H.6	Checkout re-expressed over the unified collection; group/-combo handled without a type-test ladder in the engine; numbers match Lab 4	1
Total		10

▷ Notice

A solution that passes every test but reaches the totals through four unrelated classes glued together with `instanceof` ladders — rather than one interchangeable family answering a shared call — has missed the point of the extension and will lose the design marks in H.1–H.4. The goal is not merely the right number; it is the right number produced by letting each object decide its own behaviour.

General Notes

- double for prices and multipliers; int for ages, rows, columns, counts. Round monetary totals only at the very end of checkout: `Math.round(value * 100.0) / 100.0`.

- Decide every class relationship in words — “*is a*” versus “*has a*” — before writing code. There is exactly one ownership relationship in this extension and several kinship ones; if you cannot say “is a kind of” about a pairing with a straight face, it is not kinship.
- No collection or helper imports. Fixed-size arrays, each with an `int` count, as in every prior lab.
- Partial credit is awarded; submit whatever compiles. **Submission:** commit and push to your student branch; do not commit `build/`, `.gradle/`, or `.idea/`.